SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCVCOURSE CODE: DSA02

COURSE OUTCOME COVERED

CO4: *Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)*

Assessment Tool Weightage: 30%

QUESTIONS: 2 **Total Marks:30**

Annexure A

Industry Problem Solving Task

Code of Conduct:

I B.Deepthi-192424269 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Deepthi

Question:

1: Automated Defect Detection in Manufacturing

A precision engineering company wants to automatically detect defects in mechanical components using computer vision. Parts vary slightly in shape due to manufacturing tolerances.

Questions:

1. How can PCA-based shape priors and contour-based representations be used to model standard component shapes?
2. Propose a pipeline using machine learning (SVM, GMM) or deep learning (CNN) to classify defective vs. non-defective parts.
3. Discuss methods to handle noise, partial occlusions, and varying illumination in industrial images.

2: Autonomous Vehicle Obstacle Recognition

A self-driving car company needs to detect rare or unexpected obstacles on urban roads. The labeled data for these obstacles is limited.

Questions:

1. Explain how data augmentation using GANs can help generate synthetic images for training.
2. Compare the effectiveness of CNNs versus Vision Transformers for detecting small or sparse objects in high-resolution images.
3. Suggest a real-time inference pipeline for obstacle detection, considering computational constraints in embedded systems.

1. Automated Defect Detection in Manufacturing

Industry Problem
A precision engineering company needs an automated computer-vision system to detect defective mechanical components. The main challenge is that components can have small variations in shape because of manufacturing tolerances.

1.1.How can PCA-based shape priors and contour-based representations be used to model standard component shapes?

PCA-Based Shape Priors
Principal Component Analysis (PCA) can be used to learn the normal variation of component shapes from a collection of defect-free parts.
The procedure is:

1. **Image acquisition**
   Capture images of standard, defect-free components using a fixed industrial camera and controlled lighting.
2. **Preprocessing**
   Convert images into grayscale, remove noise and apply thresholding or edge detection.
3. **Contour extraction**
   Extract the external contour of each component using OpenCV techniques such as findContours().
4. **Shape alignment**
   Align the contours using translation, rotation and scaling so that differences caused by camera position or orientation are minimized.
5. **Shape vector formation**
   Represent each normalized contour using a fixed number of points.

Using PCA for Defect Detection
If its shape can be reconstructed accurately using the learned normal shape model, it is likely to be acceptable.
The reconstruction error can be calculated as:
where:

- = actual shape
- = reconstructed PCA shape

Contour-Based Representation
Contour information is particularly useful because mechanical defects can change:

- edges,
- corners,
- dimensions,
- holes,

- boundaries
- surface profile

Combined Approach

A strong industrial solution combines **PCA shape priors with contour features**:
Camera → Preprocessing → Segmentation → Contour Extraction → Shape Alignment → PCA Features → Shape Deviation Analysis → Defect Decision
This approach is effective because PCA models **normal manufacturing tolerances**, while contour features identify specific geometric abnormalities.

1.2.Propose a pipeline using SVM, GMM or CNN to classify defective vs. non-defective parts.

A robust industrial pipeline can use a **CNN as the primary classifier**, with SVM/GMM as alternatives depending on the amount of available training data.

Proposed CNN-Based Pipeline

Industrial Camera

↓

Image Acquisition

↓

Image Preprocessing

↓

ROI Extraction

↓

Data Augmentation

↓

CNN Feature Extraction

↓

Classification Layer

↓

Defective / Non-Defective

↓

Quality-Control Decision

Step 1: Image Acquisition

Industrial cameras capture images of every component under controlled lighting. A fixed camera position helps maintain:

- consistent scale,orientation,
- focus,
- image quality.

2: Preprocessing

OpenCV can be used for:

- resizing,grayscale conversion,
- Gaussian filtering,thresholding,
- edge detection,

Step 3: Region of Interest Extraction

The component is separated from the background so that the classifier focuses on the mechanical part rather than irrelevant image regions.

Step 4: Data Augmentation

Training images can be augmented using:

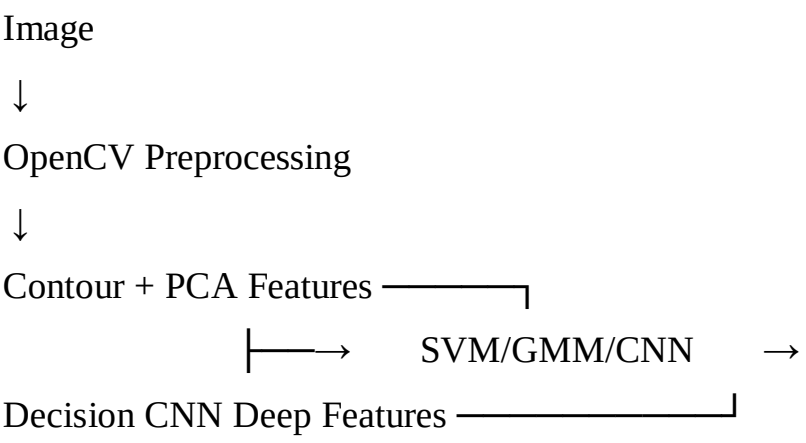
- small rotations,
- translations,
- brightness changes,

Step 5: CNN Classification

A CNN learns visual characteristics such as:

- Edges,textures,
- shapes,holes,
- cracks,missing portions,
- The final layer produces

Thus, the final system can use a **hybrid approach**:



The assessment specifically asks for an ML or deep-learning pipeline involving **SVM, GMM or CNN**, so this hybrid design directly satisfies the requirement

1.3.How can noise, partial occlusions and varying illumination be handled?

Industrial environments rarely provide perfectly clean images. Therefore, the system should be designed to handle **noise, occlusion and illumination changes**. The assessment explicitly identifies these three challenges.

A.Handling Noise

Noise can originate from:

- camera sensors,
- dust,vibrations,
- poor exposure,
- Handling Partial

Occlusion

A component may be partially blocked by:

- another component,
- mechanical equipment,
- workers,debris.

Possible approaches include:

1. Train CNNs using partially occluded examples.
2. Use data augmentation with random masking.

3. Detect visible contours instead of requiring the entire component.

C. Handling Varying Illumination

Lighting changes can cause significant differences in pixel intensity. Solutions include:

- controlled LED illumination,
- fixed camera exposure,
- background normalization,

This makes the system more reliable than depending only on a machine-learning model.

2. Autonomous Vehicle Obstacle Recognition

Industry Problem

An autonomous vehicle company needs to detect **rare or unexpected obstacles on urban roads**, but the available labeled training data for these obstacles is limited. This creates a major machine-learning challenge because deep-learning systems normally require large and diverse datasets.

2.1. How can GAN-based data augmentation generate synthetic images?

A **Generative Adversarial Network (GAN)** can generate realistic synthetic images that resemble real-world obstacle images.

A GAN consists mainly of two networks:

The conceptual process is:

Random Noise

↓

Synthetic Obstacle Image

↓

Discriminator ← Real Obstacle Image

↓

Real / Fake

As training progresses, the generator learns to create increasingly realistic obstacle images.

Application to Autonomous Vehicles

Suppose only a small number of images are available for an unusual obstacle such as an uncommon road object.

GAN-based augmentation can generate additional variations containing:

- different positions,
- lighting conditions,
- backgrounds,
- scales,
- For example:

Limited Real Images

↓

GAN Training

↓

Synthetic Images

↓

Quality Filtering

↓

Combined Dataset

↓

CNN / Transformer Training

Important Quality Control

Synthetic images should not automatically be added to the training dataset. They should be checked for:

- unrealistic object boundaries,
- incorrect geometry,

- visual artifacts,
- incorrect labels,
- unnatural road scenes.

Only high-quality synthetic samples should be included.

Benefits

GAN augmentation can:

- increase dataset size,
- improve class balance,
- represent rare scenarios,
- reduce overfitting,
- improve model robustness.

Therefore, GANs are particularly useful for the assessment's scenario where **rare obstacles have limited labeled data**.

2.2. Compare CNNs and Vision Transformers for detecting small or sparse objects in high-resolution images.

Both CNNs and Vision Transformers (ViTs) can be used for object detection, but their strengths differ.

Feature	**CNN**	**Vision Transformer**
Basic operation	Convolution	Self-attention
Local feature extraction	Excellent	Good
Global context	Limited/requires deeper layers	Excellent
Small-object detection	Strong with suitable feature pyramids	Potentially strong
Computational requirement	Generally lower	Generally higher

Training data requirement	Moderate	Often high
Embedded deployment	More practical	More challenging
Long-range relationships	Limited	Excellent
High-resolution processing	Computationally manageable with optimized architectures	Can be expensive

CNN Advantages

CNNs naturally capture local visual patterns through convolution.

For example:

Input Image

↓

CNN Backbone

↓

Multi-scale Feature Maps

↓

Detection Head

↓

Small / Medium / Large Objects

This makes CNNs attractive for real-time autonomous vehicles.

Vision Transformer Advantages

Vision Transformers divide an image into patches and use self-attention to understand relationships between different parts of an image.

- road surface,
- vehicles,
- pedestrians,
- lane markings,
- surrounding environment.

Challenge with Small Objects

Small objects occupy very few pixels. If the input is aggressively resized, important information can disappear.

Therefore, a practical high-resolution system should use:

- multi-scale features,
- high-resolution feature maps,
- suitable detection heads,
- careful image resizing.

CNN → better practical choice for real-time embedded deployment

Vision Transformer → stronger global-context modeling when computational resources permit The assessment specifically requires comparison for **small or sparse objects in high-resolution images**, making both local feature extraction and global context important

2.3.Suggest a real-time inference pipeline considering embedded-system constraints.

An autonomous vehicle needs to make decisions quickly. Therefore, the model must provide a good balance between:

- accuracy,
- latency,

- memory,
- power consumption,
- reliability.

The assessment specifically requires a real-time inference pipeline that considers computational constraints in embedded systems.

Proposed Pipeline

Camera / LiDAR / Sensors

↓

Frame Capture

↓

Preprocessing & Resizing

↓

Lightweight Object Detector

↓

Feature Extraction

↓

Obstacle Classification

↓

Confidence Filtering

↓

Multi-Object Tracking

↓

Distance / Risk Estimation

↓

Vehicle Decision System

↓

Brake / Steer / Slow Down

Step 1: Sensor Input

Cameras continuously capture road scenes.

Additional sensors such as LiDAR or radar can provide depth and distance information.

Step 2: Preprocessing

Perform:

- resizing,
- normalization,
- noise reduction,
- color conversion if necessary.

The image should be reduced to a resolution that maintains important obstacle information while reducing computation.

Step 3: Lightweight Detection Model

Use a lightweight object detector optimized for edge

devices. The model should prioritize:

- low latency,
- low memory usage,
- acceptable accuracy.

Step 4: Hardware Optimization

The trained model can be optimized through:

Quantization

Convert floating-point parameters to lower precision such as INT8 where appropriate. Benefits:

- lower memory usage,
- faster inference,
- lower power consumption.

Pruning

Remove less important model parameters.

Knowledge Distillation

Train a smaller student model using information from a larger teacher model.

Step 5: Confidence Filtering

Only detections above a suitable confidence threshold are passed to

later stages. For example:

where is the confidence threshold.

Step 6: Object Tracking

Tracking prevents the system from treating the same obstacle as a completely new object in every frame.

Tracking also provides information about:

- obstacle movement,
- direction,
- approximate velocity.

Step 7: Risk Estimation

The system estimates whether the obstacle presents a danger based on:

- distance,
- relative velocity,
- position,trajectory.

Step 8: Vehicle Decision

The final output can be:

No Hazard → Continue

Low Risk → Monitor

Medium Risk → Slow Down High Risk → Brake / Avoid

Embedded Optimization Strategy

Constraint	**Solution**
Limited CPU/GPU	Lightweight detector
Limited memory	Model compression
High latency	Quantization
Power limitations	Efficient inference
High-resolution input	Multi-scale processing
Rare obstacles	GAN-based augmentation
False detections	Confidence + tracking
Changing environments	Continuous dataset improvement